

Regularisation and Optimisation through Polynomial Fitting

FYS-STK4155 - Project 1

Esme Hookham, Lukas Monrad-Krohn, and Andrea Rosendahl

University of Oslo

(Dated: October 5, 2026)

In this project we study polynomial regression methods, resampling techniques and gradient-based optimisation using the Runge function. The Runge function provides a useful test case for polynomial regression because despite being smooth, high-degree polynomial approximations can result in strong oscillations near the interval, highlighting the challenge of balancing model complexity and generalisation. We fit polynomial models of different complexities using ordinary Least Squares (OLS), Ridge and Lasso regression, and evaluate the model performance through bootstrap and cross-validation. We further compare the closed-form method with the gradient descent to assess the tradeoff between accuracy and efficiency. The results show how increasing polynomial complexity can lead to overfitting and instability, while regularisation can improve generalisation by controlling the magnitude of the fitted coefficients. Adam converges to the Ridge closed form in 110 iterations. This model with a fifth polynomial degree and a learning rate 0.063 provides the best model for this function. Adam is the most versatile and accurate method due to the combination of the momentum and RMS. We therefore recommend using Adam with Ridge regression in this specific case.

I. INTRODUCTION

The recent boom of artificial intelligence means statistical learning methods are increasingly relevant to technological infrastructure. Despite the complexity of modern machine learning models, we can begin to understand many of their underlying ideas through studying simpler statistical models like polynomial regression. For a mathematical regression model to be useful, it should be able to make sensible predictions for data it has not seen before, as well as describe the data it has been given. This increases in importance as the model gains complexity. However, increasing the polynomial degree of the model can allow it to perfectly reproduce the training data it has been given, but also lead to overfitting, where the model has become so excessively complex it is no longer useful and begins to fit noise. The Runge function [1]

$$f(x) = \frac{1}{1 + 25x^2}, \quad x \in [-1, 1], \quad (1)$$

is a useful example of this problem since even though it is smooth and simple, high-degree polynomial approximations of it become unstable near the boundaries of the interval. The problem illustrates the bias-variance tradeoff: models that are too simple may have high bias and fail to capture the underlying function, while very flexible models have higher variance and become too sensitive to the particular training data [2]. The aim is therefore not simply to find a model that fits the training data as closely as possible, but to investigate how different choices of model complexity and regularisation affect generalisation to unseen data.

In this project, we study three linear regression methods: Ordinary Least Squares (OLS), Ridge regression and Lasso regression, and apply a bootstrap and cross-validation for assessing model performance. We construct polynomial design matrices and implement the OLS and Ridge solutions ourselves, and then develop our own numerical implementation of Lasso using gradient-based optimisation. We additionally compare gradient-based optimisation methods, using the closed-form regression solutions as reference points for evaluating convergence. We then extend the basic method with momentum [3], AdaGrad [4], RMSprop [5] and Adam [6], comparing their convergence behaviour and dependence on the learning rate. We also implement stochastic gradient descent, where the training data is divided into minibatches and the parameters are updated repeatedly over several epochs [7]. Automatic differentiation is provided through a library implementation [8], while the regression cost functions and optimisation procedures are constructed and tested independently.

The report is organised as follows: Section 2 introduces the models and methods. Section 3 is where our results are described and discussed, and the reasons for our final model selection are described in section 4. Lastly, a conclusion is presented in section 5. Mathematical derivations and additional results are provided in the appendices. The code is available at <https://github.com/lukasMK01/FYS-STK4155/>.

II. METHODS

The polynomial degree and the regularisation parameter are variable in our code to help us best compare the models.

A. Scaling and preprocessing

Polynomial regression can produce features with substantially different numerical scales. For example, the columns x , x^2 and x^d can have different variances, and this becomes increasingly important as the polynomial degree is increased. Scaling is therefore used to improve the numerical behaviour of the regression and optimisation procedures and to ensure that regularisation acts on coefficients corresponding to features of comparable scale. The predictor matrix is centred and standardized column-wise according to

$$X_{ij}^{\text{centred}} = \frac{X_{ij} - \bar{X}_j}{\sigma_j}, \quad (2)$$

where $\bar{X}_j$ and σ_j are the mean and standard deviation of feature j . The y-values are centred by subtracting their mean,

$$y^{\text{centred}} = y - \bar{y}. \quad (3)$$

In the initial OLS and Ridge experiments, the polynomial design matrices are centred and standardized before fitting. The main design matrices are constructed without an intercept column, consistent with the use of centred y-values. In the later cross-validation experiments, `StandardScaler` from Scikit-Learn is used within the regression pipeline. The scaler is fitted on the training portion of each fold and then applied to the corresponding validation fold, ensuring that the validation data does not influence the preprocessing. The polynomial degree and regularisation parameter are treated as variable model settings throughout the analysis. This allows the effect of model complexity and regularisation to be examined systematically before the final model comparison using cross-validation. The bias-variance decomposition underlying the evaluation of model complexity and generalisation is derived in Appendix C.

B. Regression methods

The aim of the regression analysis is to approximate Runge's function using polynomial models of increasing degree. The data is generated by evaluating the function at points sampled uniformly from the interval $[-1, 1]$ and adding normally distributed noise,

$$y_i = f(x_i) + \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, \sigma^2), \quad (4)$$

and unless otherwise stated, the calculations use $n = 100$ observations and a noise level of $\sigma = 0.1$. For selected experiments, the number of observations and noise level is varied in order to investigate the differing effect on the regression results.

1. Ordinary Least Squares

We use OLS as the baseline regression method. The parameters are obtained by minimizing the mean squared error between the observed and predicted values, in one precise calculation. The analytic closed-form OLS solution is

$$\hat{\theta}_{\text{OLS}} = (X^T X)^{-1} X^T y. \quad (5)$$

A derivation for which can be found in Appendix B. Rather than explicitly calculating the inverse of $X^T X$, our main OLS implementation uses the singular value decomposition (SVD) of the design matrix,

$$X = U \Sigma V^T. \quad (6)$$

The parameters are then calculated from the singular values and singular vectors according to

$$\hat{\theta}_{\text{OLS}} = V \Sigma^{-1} U^T y. \quad (7)$$

The SVD formulation avoids directly forming the matrix inverse and is particularly useful when the design matrix becomes poorly conditioned for high polynomial degrees. For the main polynomial-degree analysis, OLS is fitted for degrees from 1 to 14. For each degree, the fitted parameters are used to calculate predictions on both the training and test data, and the resulting MSE and R^2 scores are recorded. The fitted parameter values are also stored as the polynomial degree is increased in order to examine the behaviour of the regression coefficients. In the main OLS implementation, the polynomial design matrix is constructed without an explicit intercept column, the x-values are centred by the mean and scaled by the standard deviation and the y-values are centred before fitting. The effect of increasing the number of observations is investigated separately using datasets with $n = 40, 100$ and 400 observations.

2. Ridge and Lasso Regression

We introduce shrinkage to OLS to help manage its higher variance, and therefore higher prediction error. Ridge regression extends OLS by adding an L_2 regularisation term which penalizes large regression coefficients. The cost function is written as

$$C(\theta) = \frac{1}{n} \|y - X\theta\|_2^2 + \lambda \|\theta\|_2^2, \quad (8)$$

where $\lambda \geq 0$ controls the strength of the regularisation. With this normalisation, the corresponding closed-form solution is

$$\hat{\theta}_{\text{Ridge}} = (X^T X + n\lambda I)^{-1} X^T y. \quad (9)$$

The factor of n depends on the normalisation convention used for the cost function and is kept consistent with the convention used in the gradient-descent calculations. Increasing λ increases the penalty on large coefficients and therefore shrinks the fitted parameters towards zero. We varied both the polynomial degree and the regularisation parameter λ , calculating training and test MSE for several combinations of degree and λ .

The Ridge results are compared with the corresponding OLS results to study the effect of coefficient shrinkage and regularisation on polynomial fits.

Lasso regression introduces an L_1 regularisation term instead of the L_2 penalty used by Ridge. Its cost function is

$$C(\theta) = \frac{1}{n} \|y - X\theta\|_2^2 + \lambda \|\theta\|_1, \quad (10)$$

where

$$\|\theta\|_1 = \sum_j |\theta_j|. \quad (11)$$

The L_1 penalty differs from the Ridge penalty because it can cause individual regression coefficients to become exactly zero. Lasso can therefore provide a sparse representation of the polynomial model. Unlike OLS and Ridge, Lasso has no ordinary closed-form solution because the absolute-value function is not differentiable at zero. In this project, Lasso is therefore solved numerically using gradient-based optimisation methods. The implementation applies an iterative parameter update followed by a soft-thresholding operation,

$$S_{\frac{\lambda}{2}}(z) = \text{sign}(z) \max\left(|z| - \frac{\lambda}{2}, 0\right), \quad (12)$$

which promotes zero-valued coefficients. The numerical results are compared with the corresponding Scikit-Learn Lasso implementation as a reference.

C. Evaluation and resampling techniques

In each application we use 70% of the data for training and 30% for testing, to achieve independent results. We use the MSE and the R^2 -score to evaluate the error between each method and a reference. Resampling methods are used to test how sensitive the fitted models are to the specific training data used. In this project, bootstrap resampling is used to study the bias-variance tradeoff, while k -fold cross-validation is used to compare model complexity and regularisation parameters across different regression methods.

The bias-variance decomposition provides a framework for understanding the effect of model complexity on prediction error, with expected test error consisting of squared bias, variance and irreducible noise (see Appendix C). Increasing model complexity can reduce bias by allowing the model to represent the underlying function more closely, but may also increase variance by making predictions more sensitive to the particular training data. Bootstrap resampling is therefore used to examine how the variability of the fitted models changes with polynomial degree.

1. Error metrics

The choice of error metrics is wide. Here we use the MSE because it penalizes large errors by squaring the difference and the square prevents cancellation of positive and negative errors.

$$MSE(\mathbf{y}, \tilde{\mathbf{y}}) = \frac{1}{n} \sum_{i=0}^{n-1} (y_i - \tilde{y}_i)^2. \quad (13)$$

The coefficient of determination (R^2) describes how well a statistical model explains the variance of a dependent variable

$$R^2(\mathbf{y}, \tilde{\mathbf{y}}) = 1 - \frac{\sum_{i=0}^{n-1} (y_i - \tilde{y}_i)^2}{\sum_{i=0}^{n-1} (y_i - \bar{y})^2}, \quad (14)$$

where the mean value of $\mathbf{y}$ is $\bar{y} = \frac{1}{n} \sum_{i=0}^{n-1} y_i$. It usually ranges between 0 and 1, but can be negative when predictions are worse than the average of the target values.

2. Bootstrap

Bootstrap resampling estimates the bias and variance by repeatedly drawing training sets of size m with replacement from the original training data. For each of the $B = 100$ bootstrap samples, the regression model is fitted and used to predict the same test set, giving predictions $\tilde{y}^{(b)}$ for $b = 1, \dots, B$. The bootstrap mean prediction is then

$$\mathbb{E}[\tilde{y}] \approx \frac{1}{B} \sum_{b=1}^B \tilde{y}^{(b)}, \quad (15)$$

from which the prediction error, bias and variance are estimated. The expectation is therefore taken over the resampled training sets, while the test data are kept fixed. The analysis is performed for different polynomial degrees and numbers of observations to investigate the bias-variance tradeoff. For the bias estimate, the unknown $f(x)$ is replaced by the observed y , causing the measured bias term to also contain the noise variance σ^2 .

3. k -fold cross-validation

In k -fold cross-validation, the dataset is randomly shuffled and divided into k subsets. For each fold, one subset is used for validation while the remaining $k - 1$ subsets are used to fit the model. The resulting validation MSE is averaged over all folds,

$$MSE_{CV} = \frac{1}{k} \sum_{j=1}^k MSE_j. \quad (16)$$

We use $k = 5$ and $k = 10$ to evaluate OLS and Ridge as functions of polynomial degree and, for Ridge, the regularization parameter λ . Scikit-Learn's `KFold` and `cross_val_score` functions are used for the main calculations, while an independent k -fold implementation is used to verify the results for selected parameter combinations. Scaling is performed within each fold by fitting `StandardScaler` on the training data only and applying the same transformation to the validation data.

D. Gradient-based optimisation

1. Gradient descent

Gradient descent is a deterministic method that employs the idea that a function decreases most quickly in the direction of the negative gradient [2], and is given by

$$\theta_{k+1} = \theta_k - \gamma \nabla_{\theta} C(\theta_k) \quad (17)$$

where θ_{k+1} is the new estimated parameters, θ_k is the current estimated parameters, γ is the learning rate and $\nabla_{\theta} C(\theta_k)$ is the gradient of the cost function, given by

$$\nabla_{\theta} C = \frac{2}{n} \mathbf{X}^T (\mathbf{X}\theta - y) + 2\lambda\theta \quad (18)$$

where n is the number of data points, $\mathbf{X}$ is the design matrix, y is the target value, and λ is the penalty parameter (0 for OLS). This method is a form of iterative optimisation, where new sets of parameters are estimated at each iterations, until the method reaches a convergence to a selected tolerance or all iterations are used. The gradient of the cost function can be written both analytically (see eq. 18) and by using automatic differentiation (using JAX functions [8]). The two gradients agree to machine precision for both OLS and Ridge, confirming that the analytical equation is correctly written.

Automatic differentiation (AD) is neither symbolic nor numerical (finite-difference) differentiation. A symbolic differentiation is exact and it calculates the formula for f' from the formula for f [2]. This type of differentiation requires a single closed-form expression, which can result in a large size with many terms. The AD, however, calculates the derivative at a point instead of using a formula, thereby escaping the possible large size. Further, a numerical differentiation means that the derivative is replaced by a difference quotient [2]. Here, there is a truncation error that depends on size of step h . The AD uses exact derivations, and does not have any step h causing a truncation error. The cost of the AD is original analytical cost.

One limitation connected to the gradient descent is that we do not know if we find a local or global minimum. This depends on the initial condition and the learning rate γ . The gradient descent is sensitive to the learning rate, as a too large learning rate can result in diverging gradient descent, while too low learning rate requires many iterations to converge to a tolerance level. The maximum possible (fixed) learning rate γ_{max} is $< 2/\sigma_{max}$, where σ_{max} is the maximum eigenvalue of the Hessian matrix, given by

$$H = \frac{2}{n} \mathbf{X}^T \mathbf{X} + 2\lambda \mathbf{I} \quad (19)$$

A function is convex when the Hessian matrix is semi-positive definite (such as OLS linear regression), and it is strictly convex when the Hessian matrix is positive definite (Ridge when $\lambda > 0$) [2]. This means that the minimum is global. There is no dependence on θ in the Hessian matrix for OLS, Ridge and Lasso regression. The Hessian indicates the speed of convergence for the gradient descent, through the condition number

$$\kappa = \sigma_{max}/\sigma_{min} \quad (20)$$

which is the ratio of the largest and smallest eigenvectors from the Hessian matrix. A large condition number suggests therefore that the convergence is slow and it is necessary with a lot of iterations, while a small condition number suggests a rapid convergence.

2. Momentum and adaptive optimisers

The gradient descent can be extended to include methods that iteratively update the learning rate: Momentum, AdaGrad, RMSProp and Adam.

The momentum gradient descent is given by

$$vk + 1 = \beta vk - \gamma \nabla_{\theta} C(\theta_k) \quad (21)$$

where β is the momentum parameter, given as 0.9 in this analysis. Here, the iterative update is based on all previous gradients through an exponentially weighted sum [2]. Momentum is accelerated when the gradient is moving in a consistent direction, and reduces the oscillations caused by rapidly changing gradient directions. It can therefore travel past local minima in favour of more useful global minima.

AdaGrad gradient descent includes a running sum of the squared gradients [2], and is given by

$$\theta_{k+1,j} = \theta_{k,j} - \frac{\gamma}{\sqrt{r_{k,j}} + \epsilon} g_{k,j} \quad (22)$$

where $g_k = \nabla_{\theta} C_k(\theta_k)$ and $r_{k,j} = r_{k-1,j} + g_{k,j}^2$. We include a small constant $\epsilon = 1e - 8$ to avoid division by 0. The learning rate decreases as $r_{k,j}$ increases, which means that the learning rate decays iteratively. This can be a limitation to large problems, if the learning rate becomes substantially small before the minimum is found. Further, the memory in $r_{k,j}$ can be infinitely large.

RMSProp gradient descent is given by

$$\theta_{k+1} = \theta_k - \frac{\gamma}{\sqrt{v_k} + \epsilon} \nabla_{\theta} C(\theta_k) \quad (23)$$

which includes an exponentially decaying weighted mean $v_k = \rho v_{k-1} + (1 - \rho)(\nabla_{\theta} C(\theta_k))$ [2]. Here, we use $\rho = 0.99$, and the same $\epsilon = 1e - 8$ as for AdaGrad. In contrast to AdaGrad, the v_k effectively forgets old gradients and is estimated on only the recent gradients. Therefore, the learning rate does not decay to zero as in AdaGrad. However, a limitation to RMSProp is that once the gradient descent is close to the minimum, the gradient is small and the learning rate stops shrinking, possibly preventing convergence and a small enough tolerance level.

Adam (adaptive moment estimation) gradient descent is given by

$$\theta_{k+1} = \theta_k - \frac{\gamma}{\sqrt{\hat{v}_k} + \epsilon} \hat{m}_k \quad (24)$$

where $\hat{v}_k = \frac{v_k}{1 - \beta_2^k}$ is the RMS term (second moment) and $\hat{m}_k = \frac{m_k}{1 - \beta_1^k}$ is the momentum term (first moment). This method therefore combines both momentum and RMSProp with running averages of the first and second moment of the gradient [2]. Here, $\beta_1^k = 0.9$ and $\beta_2^k = 0.999$. These factors are deterministic, and works as a bias correction through rescaling. The correction is large at few iterations and becomes iteratively smaller until the moments become unbiased.

3. Stochastic gradient descent

Stochastic gradient descent reduces the cost related to computing the full gradient. The method is cheaper as we calculate the gradient on a subset of the data instead of the full dataset. We randomly split the data into a set of mini-batches with size M . Here, we select $M = 1, 5, 25$ and 100 for the analysis. $M = 100$ is equal to the full-batch gradient, as we have $n = 100$ data points. Increasing the size of the mini-batches decreases the noise of a mini-batch with $1/\sqrt{M}$. We use 100 epochs for our analysis. This results in $100 * 100 = 10000$ evaluations (epochs*datapoints n).

Additionally, the stochasticity implies an addition of noise, which increases the likelihood of finding a global minimum. However, the method does not monotonically converge to a minimum but reaches a plateau that is dependant on the noise and the learning rate. The convergence is therefore sensitive to the learning rate, and one way to improve the distance to the closed-form solution is by including scheduled learning rates. We include a decaying learning rate, given by the schedule $\frac{t_0}{t+t_1}$, and compare this to constant learning rates.

E. Implementation

The data is generated from the Runge function (Equation 4) with 100 random samples (seed=2026) within the range of $[-1,1]$. We add artificial noise that is normally distributed with a standard deviation of 0.1.

```
def generate_data(n=100, sigma=0.1, seed=2026):
    rng = np.random.default_rng(seed)
    x = np.sort(rng.uniform(-1, 1, n))
    y = runge(x) + rng.normal(0, sigma, n)
    return x, y, n, sigma
```

Based on this data we prepare the design matrix without intercept in the function `design_matrix.prepare_design_matrix` centres and scales the train and test split data as described in Section II A.

Most of the equations discussed in Section II can be represented in multiple ways or with and without using existing packages and functions. This is how we implemented key functions for this project. The full code is available on <https://github.com/lukasMK01/FYS-STK4155/>. There is also a `.yaml` file to set up an environment in python version 3.13.13. The main packages used are numpy [9] for basic operations and arrays, scikit-learn [10] for error metrics, train-test split, OLS, Ridge, and Lasso implementations, as well as k-fold cross-validation. We used JAX [8] for automatic-differentiation. The closed form of OLS is our reference for evaluating OLS methods that are more computationally efficient for large datasets. We calculate it using the `closed_form` function

```
def closed_form(X, y, lam=0.0):
    n, p = X.shape
    return np.linalg.solve(X.T@X + n*lam*np.eye(p), X.T @ y)
```

where `lam` is 0 for OLS and have compared this method with a solution based on singular value decomposition (not shown). We use the regularization convention $n \cdot \lambda$. In cross-validation and bootstrap the OLS solution is calculated using scikit-learn `model_selection`, `preprocessing`, `pipeline` and `utils`.

The `closed_form` lets us calculate the Ridge regression as well, using `lam` $\neq$ 0. When we apply bootstrap or cross-validation methods to the Ridge regression, scikit-learn's Ridge class provides the necessary functions to calculate the Ridge regression as follows:

```
theta = Ridge(alpha=1, fit_intercept=False).fit(X_train_norm, y_train_norm).coef_.
```

The gradient descent method is implemented using the equations discussed in Section II D 1. The `gradient_descent` includes an optimiser step to adjust the learning rate γ and a soft threshold, if Lasso regression is chosen. Additionally, one can choose between calculating the gradient using analytical methods or by automatic differentiation. These methods have been verified that they produce the same results.

```
def gradient_descent(X,y,gamma,lmbda=0.0, num_iters=1000, tol=1e-8,
                    method="plain",reg_method="lasso",grad_method="analytical"):

    # Initialize weights for gradient descent
    n_features = X.shape[1]
    theta0 = np.zeros(n_features)
    history = [theta0.copy()]
    theta_an, state = np.array(theta0, dtype=float), {}

    # Gradient descent loop
    for t in range(1, num_iters+1):

        if grad_method == "analytical": # the gradients by hand (analytically)
            g = gradient(theta_an, X, y, lmbda)
        elif grad_method == "AD": # gradients by automatic differentiation
            g = grad_jax(theta_an, X, y, lmbda)

        # Update parameters theta, depending on method:
        theta_an, state = optimiser_step(method, theta_an, g, state, t, gamma, beta=0.9, rho=0.99,
                                         beta1=0.9, beta2=0.999, eps=1e-8)

        # Soft threshold (if Lasso)
        if reg_method == "Lasso":
            theta_an = soft_threshold(theta_an, lmbda)

        history.append(theta_an.copy())

    # Stopping criterion: gradient is small enough
    if np.linalg.norm(g) < tol:
        break

    return theta_an, t, history
```

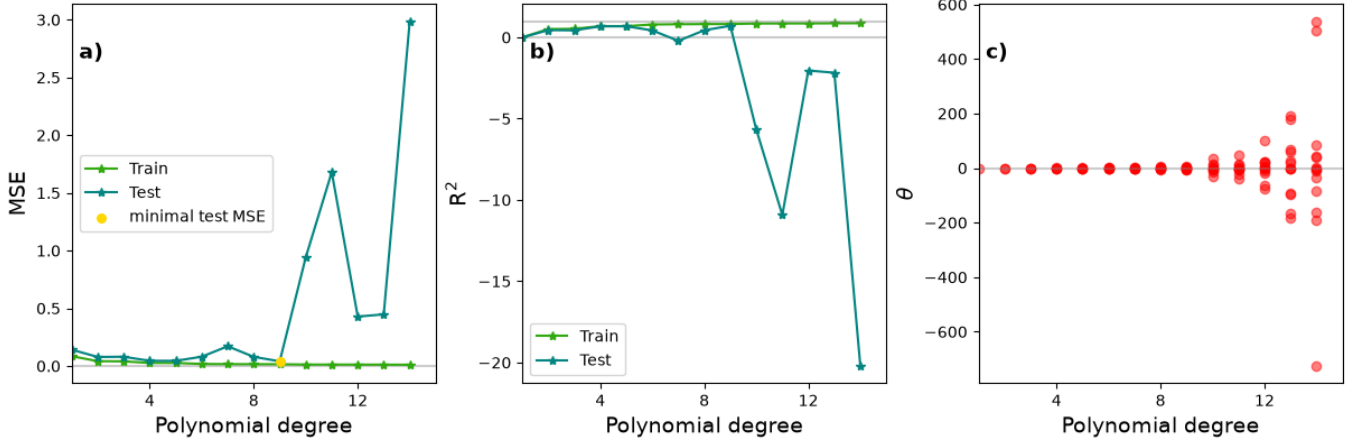


Figure 1: (a) MSE and (b) R^2 of the test and train datasets for the OLS fit of varying complexity. (c) shows the θ parameters calculated through OLS.

The stochastic gradient descent needs a separate function `sgd` to split into mini-batches and run the gradient descent for each batch.

```
def sgd(X, y, method="plain", n_epochs=50, batch_size=5, gamma=0.1, schedule=None,
        lam=0.0, seed=2026, theta0=None, **kw):
    rng = np.random.default_rng(seed)
    n, p = X.shape
    theta = np.zeros(p) if theta0 is None else np.array(theta0, dtype=float)
    state, t, history = {}, 0, [theta.copy()]
    for epoch in range(n_epochs):
        for batch in make_batches(n, batch_size, rng):
            t += 1
            g = gradient(theta, X[batch], y[batch], lam)
            g_t = gamma if schedule is None else step_length(t, *schedule)
            theta, state = optimiser_step(method, theta, g, state, t, g_t, **kw)
        history.append(theta.copy())
    return np.array(history)
```

This requires that the batches are created randomly, through the `make_batches`. Additionally we include an optional scheduled learning rate that is a decaying schedule `step_length`.

```
def make_batches(n, batch_size, rng):
    idx = rng.permutation(n)
    return [idx[i:i + batch_size] for i in range(0, n, batch_size)]

def step_length(t, t0, t1):
    return t0 / (t + t1)
```

III. RESULTS AND DISCUSSION

A. OLS and Ridge regression

In any of these experiments we expect that the error of the training data decreases with increasing polynomial degree of the fitted function, because it can then reproduce more of the small-scale features and noise. The MSE of the training data converges to 0 for increasing polynomial degrees (Fig. 1 a) and the R^2 converges towards 1 (Fig. 1 b), signalling improved representation of the Runge function and noise. However, the performance of the regression on unseen data is the relevant goal. Both the MSE and R^2 indicate that the best performance at polynomial degrees 9, 5 and 4 in that order. At polynomial degree ≥ 10 the fit yields unstable results with a magnitude higher MSE and negative R^2 . At these degrees the model overfits the oscillations at the edges of the domain for which we apply the

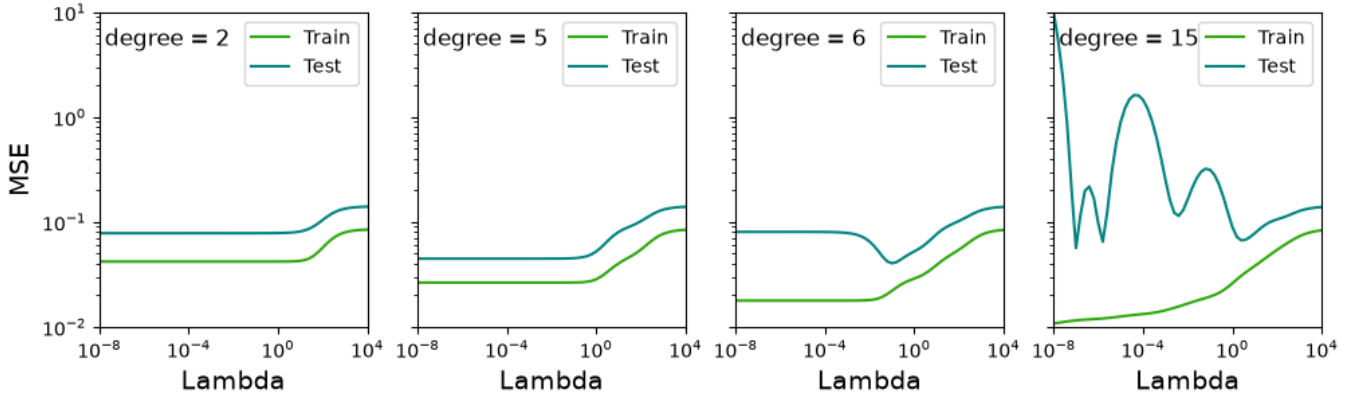


Figure 2: The MSE of the Ridge regression applied to the Runge function is evaluated for varying penalty parameter λ at polynomial degrees 2, 5, 6 and 15.

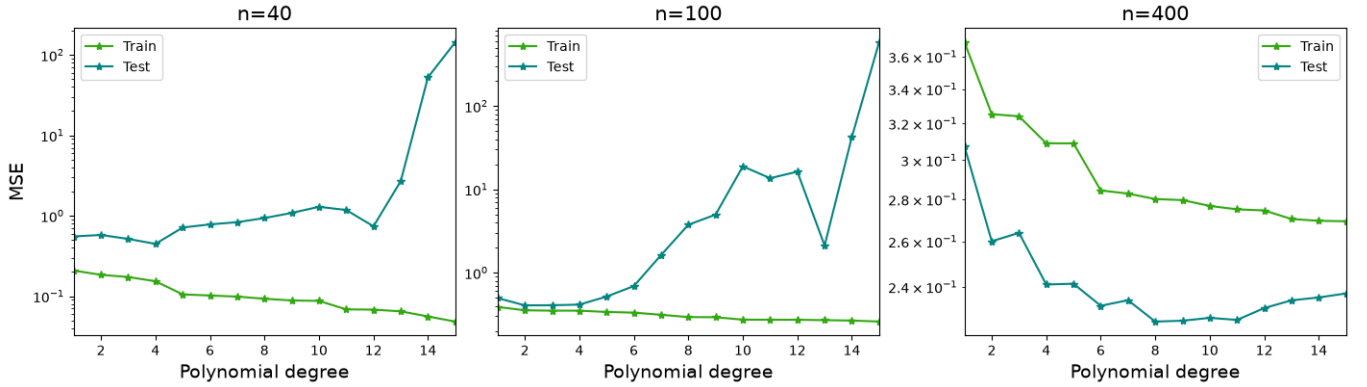


Figure 3: The MSE of the OLS regression on the Runge function at different complexity for datasets of different sizes ($n=40$, 100, 400). Following the idea of Hastie et al. [11].

Runge function. These clear signals in MSE and R^2 are caused by the increasing variability of the coefficients θ (Fig. 1 c), as they represent more details of the function. Large positive and negative coefficients cancel each other out, which is why it still represents the training data well. Although the fifth degree provides the second lowest MSE, it is the best solution when applied to a large dataset, because the difference to the ninth degree is small and the fifth degree is computationally cheaper than ninth.

We apply the same method using Ridge regression and compare the train and test MSE for selected polynomial degrees as a function of the penalty parameter λ (Fig. 2). The MSE of the training data is similar for large penalty parameters, but for low penalty parameters the training MSE decreases with increasing polynomial degree. This is the same as for OLS, because it approaches the OLS for small penalties. The MSE of the independent test data shows similar behavior as the training data for large penalty parameters at every polynomial degree, but differ for $\lambda < 1$. The fifth degree Ridge regression provides the lowest MSE for a range of penalty parameters and from degree 6 and higher similarly good results are only possible for specific and small ranges of penalty parameters. The high MSE at small penalty parameters for degrees six and higher are caused by overfitting, as the shrinkage effect is small and the Ridge and OLS methods are close to each other. For large penalty parameters we see underfitting, as the real coefficients shrink too much and the real signal is dampened. At moderate penalty parameters, for example around 0.1 at sixth degree, the penalty parameter effectively shrinks the coefficients to maintain the signal.

Furthermore, the ideal polynomial degree of a regression also depends on the size of the dataset, as we show in Fig. 3 using the OLS. At $n = 40$ and $n = 100$ the best performance is at degree 4 and 3, respectively. For $n = 400$, OLS achieves the best result at degree 8. However, the MSE is near constant for varying polynomial degree compared to the smaller datasets. Using a larger dataset and polynomial degree increases computational costs in two ways. It is surprising that the error of the training dataset is higher than the test dataset. In this case it is possibly caused by a lucky train-test split of the data, with favourable noise in the test data.

Using the bootstrap resampling method (Section IIC2) we decompose the total MSE into bias, variance and test

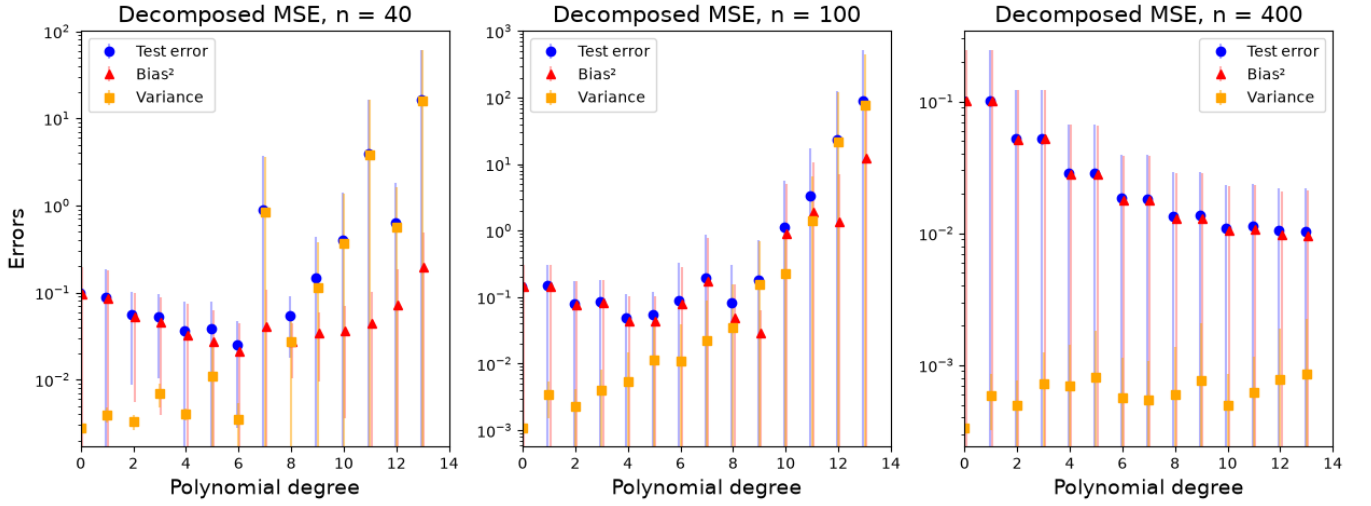


Figure 4: Similar to Fig. 3, but the MSE is decomposed into the test error, bias and variance using the bootstrap resampling method.

λ	Error	Bias ²	Variance
OLS	0.0762	0.0284	0.0478
0.0001	0.0338	0.0272	0.0067
0.1	0.0289	0.0228	0.0060
10	0.1020	0.0979	0.0041

Table I: Test error decomposed into squared bias and variance for a degree-12 polynomial with $n = 40$ data points and 100 bootstrap samples, for OLS and Ridge regression at different values of λ .

error for $n = [40, 100, 400]$ (Fig. 4). Sample sizes of 40 and 100 produce similar results in that the bias and test error follow a U-shape with a minimum at the sixth polynomial degree, whereas the variance increases with polynomial degree. For small degrees, the bias is the largest contributor to the test error, but at high degrees the variance is dominant. For $n = 40$ and $n = 100$ the best combination of bias and variance and thus lowest test error are at degree 6 and 4, respectively. The large dataset of $n = 400$ has nearly constant variance and decreasing bias, which indicates that the optimal degree is at degree 13 or higher. Aside from the polynomial degree, we see that the variance becomes more dominating for smaller sample sizes. When doing the same analysis with Ridge regression at fixed degree 12 (Table I), the penalty parameter influences both bias and variance and we achieve the best results with the lowest

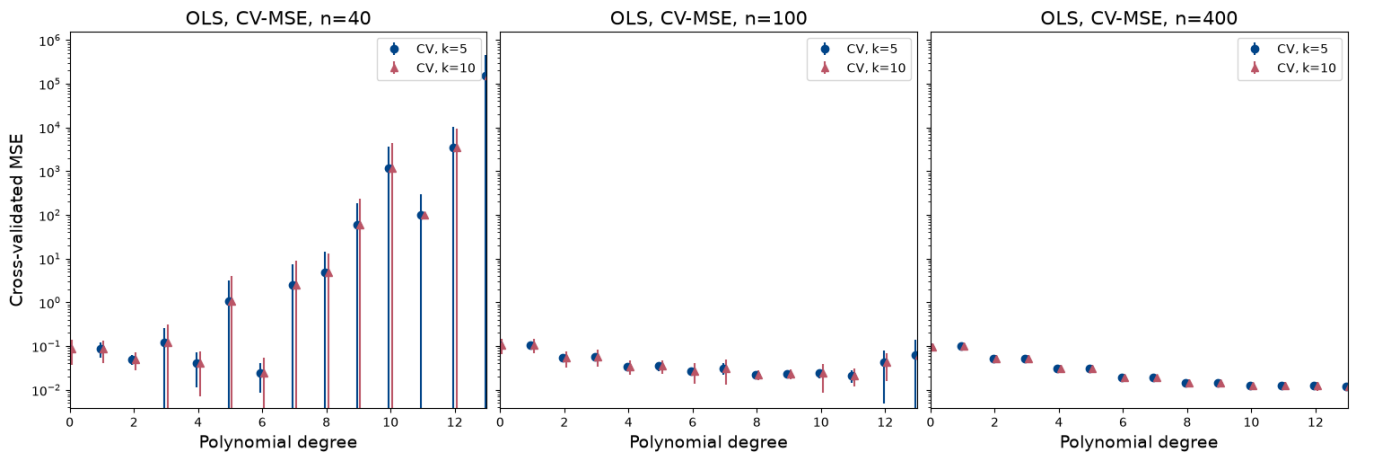


Figure 5: Mean cross-validated MSE for $k=5$ and $k=10$ for the OLS regression of varying complexity and different sample sizes. The error bars mark the standard deviation.

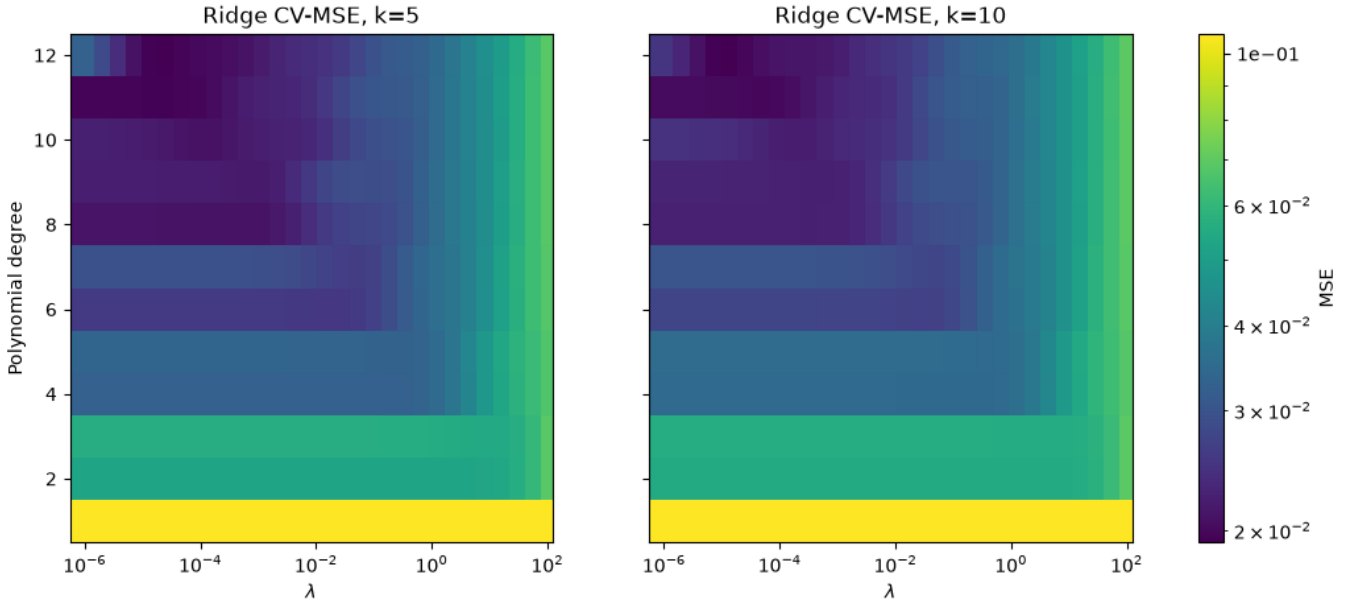


Figure 6: Mean cross-validated MSE for $k=5$ and $k=10$ for the Ridge regression of varying complexity and λ .

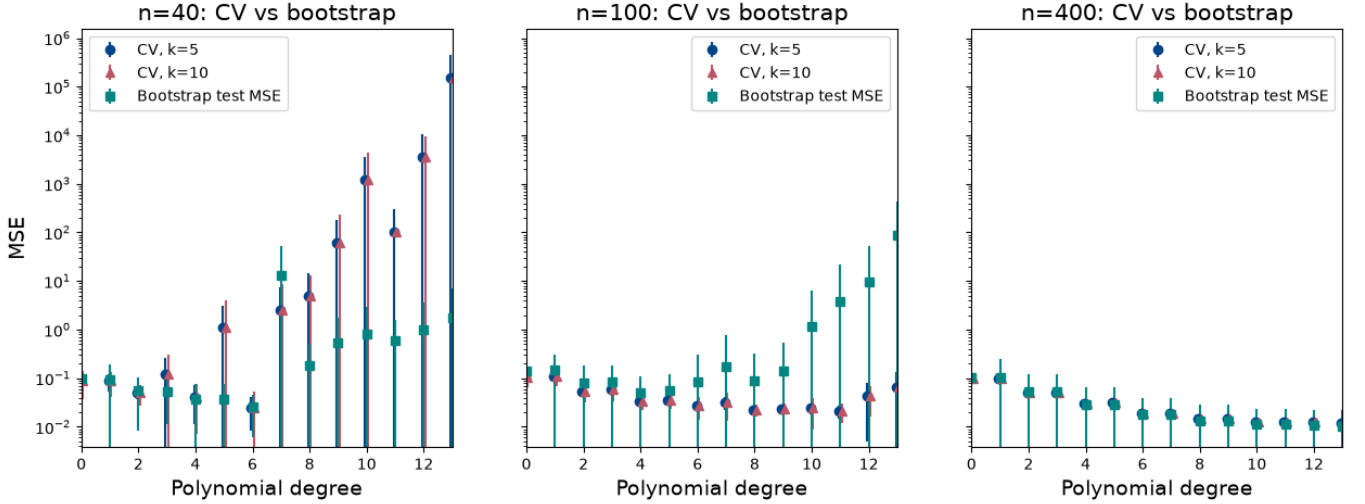


Figure 7: Comparison of the cross validated MSE from Fig. 5 and the bootstrap method from Fig. 4.

bias and variance for $\lambda = 0.1$.

Another way of assessing the MSE is through k -fold cross-validation (Section II C). We implement two sets of number folds at $k = 5$ and $k = 10$ for datasets of three different sizes for the OLS regression (Fig. 5). Across all of these parameter combinations, both numbers of folds produce similar results. With increasing sample size, the standard deviation decreases, the MSE decreases and becomes less sensitive to the number of polynomial degrees. All different sample sizes show the characteristic U-shape of the bias-variance tradeoff and similar to the other methods the optimal polynomial degree increases from 6 at $n = 40$ to 8 at $n = 100$ and 13 or larger at $n = 400$.

When applying k -fold cross-validation to Ridge regression on a dataset of $n = 100$ (Fig. 6), we would expect the optimal value to be similar as in Fig. 2 at polynomial degree 5 and around $\lambda = 0.1$, but instead it is at polynomial degree 11 and penalty parameters around $\lambda = 1e - 4$. The results are similar, independent of the number of folds k . A similarity to the closed form Ridge regression is the dependence on the penalty parameter for constant polynomial degree. At low degrees, the MSE is constant for low penalties and increases above a certain penalty, which decreases with polynomial degree. To evaluate the cross-validation implementation, we compare our own code with the scikit-learn method, which produces the same MSE.

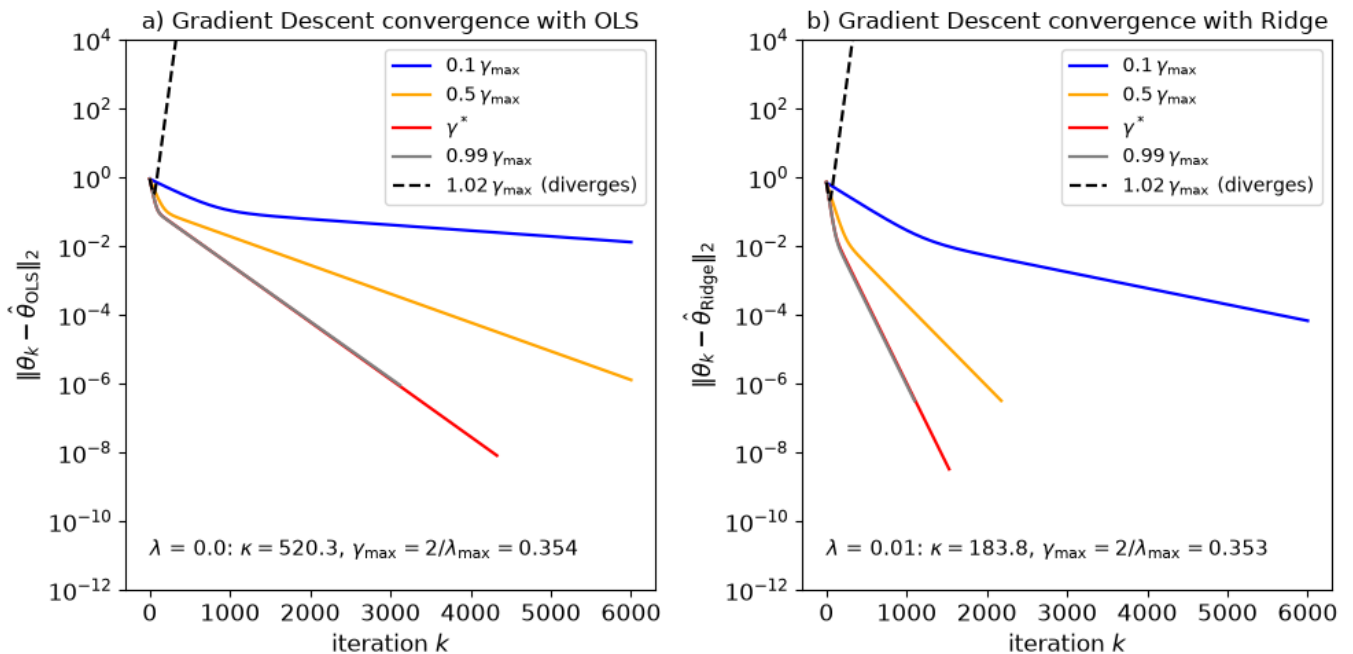


Figure 8: Gradient descent convergence to the closed-form solution using (a) OLS and (b) Ridge ($\lambda = 0.01$) regression for 5 selected learning rates γ . The optimal learning rate (red) converges in the fewest number of iterations k , while learning rates above the maximum diverges (black hatched line).

As k -fold cross-validation and bootstrap are both methods that repeatedly apply the regression methods to subsets of the data, we validate them against each other at different polynomial degrees and dataset sizes (Fig. 7). At $n = 40$ the mean values are similar for low polynomial degrees, but diverge for larger degrees. However, both find the sixth degree to yield the optimal combination of bias and variance. With increasing sample size bootstrap and cross-validation converge, but the largest differences are at higher polynomial degrees and cross-validation performs better. For the dataset of $n = 100$ they find different optimal polynomial degrees. However, these are not representative as the sensitivity to the polynomial degree is low and the standard deviation large. At $n = 400$ they are indistinguishable and no overfitting is seen up to degree 13. The optimal complexity increases with sample size, as expected since more data allows more parameters to be constrained.

In the further experiments using Ridge regression we use $\lambda = 0.01$, as it yielded the best result for Ridge regression of degree 5 (Fig. 2). Although the closed form disagrees with the results of the k -fold cross-validation (Fig. 6), we choose it, because of its accuracy, speed and reproducibility.

B. Gradient descent methods

While the closed-form solutions produce the ideal minimum, calculating the analytical solution can be costly for large datasets with many dimensions. We therefore apply the gradient descent (eq. 17), which is a cheaper approximation of the minimum, on the Runge function to study the skill and speed compared to the closed-form solutions.

The gradient descent converges to the closed form solutions of both OLS and Ridge (with a tolerance of $1e-8$) for selected learning rates (Fig. 8). The Ridge regression converges at much fewer iterations than the OLS regression. For the optimal learning rate ($\gamma^* = 0.353$ for OLS, $\gamma^* = 0.351$ for Ridge; Fig. 8 red lines), the gradient descent converges after 4327 iterations for OLS and after 1529 iterations for Ridge. This is supported by the condition number κ (eq. 20), which is 520.3 and 183.8 for OLS and Ridge, respectively. Fewer iterations for Ridge is as expected, as Ridge is generally more efficient due to the penalising factor (λ). We tested the gradient descent convergence by applying both the gradient by automatic differentiation and the analytical gradient. The two gradient methods produce the same results.

The learning rate plays an important role in how fast the gradient descent converges. The largest learning rate for which the gradient descent converges is $\gamma_{max} = 2/\lambda_{max}$, where λ_{max} is the maximum eigenvalue from the Hessian matrix (eq. 19). The maximum learning rate is 0.354 and 0.353 for OLS and Ridge respectively, suggesting the chosen

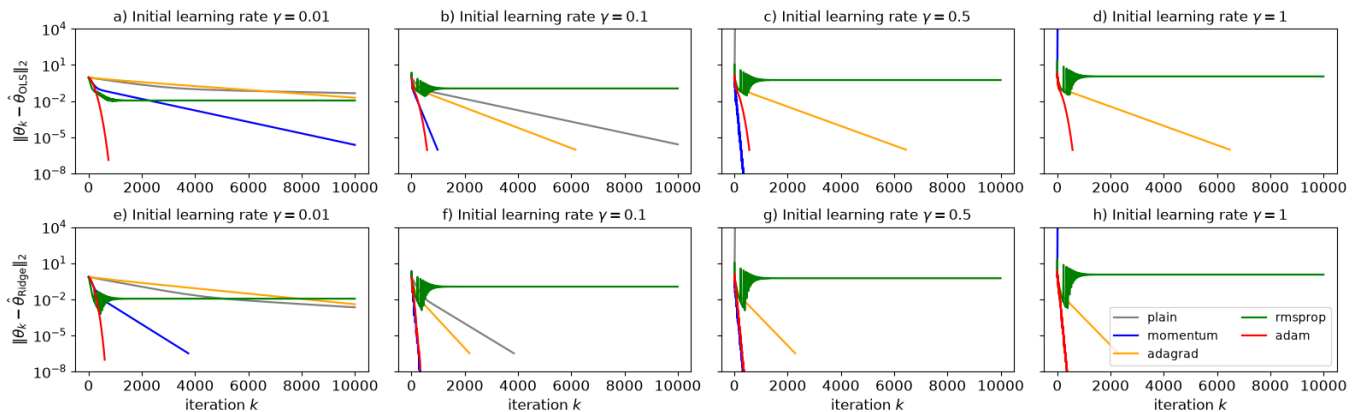


Figure 9: Gradient descent convergence to the closed-form solution for (upper panel) OLS and (lower panel) Ridge regression using five different methods for the learning rate: plain (gray), momentum (blue), adagrad (yellow), rmsprop (green) and adam (red). The number of iterations for convergence (if the method converges) depends on the initial learning rate γ .

penalty in Ridge ($\lambda = 0.01$) has little effect on the range of possible learning rates. Fig. 8 shows that the optimal learning rate γ^* is almost equal to the maximum γ_{max} . Lower learning rates, however, need more iterations for the gradient descent to converge to the closed form. In contrast, the gradient descent diverges when the learning rate is above the maximum learning rate (black hatched lines).

The plain gradient descent uses a constant learning rate, which is not ideal as a too large learning rate will jump over the minimum and a too small learning rate takes too many iterations to converge. We therefore want to improve our gradient descent by including four methods that recurrently update the learning rate: Momentum, AdaGrad, RMSProp and Adam. We also keep the method from earlier that uses a constant learning rate and refer to it as *plain*, resulting in five methods. Fig. 9 shows the gradient descent method relative to the OLS and Ridge closed-form solutions for each of the five methods. A higher initial learning rate γ generally reduces the number of iterations needed for convergence. However, some of the methods start diverging at too high learning rates. This is similar to what is seen in Fig. 8, where the optimal learning rate was close to the maximum possible learning rate. Some of the methods diverge at selected initial learning rates, while methods such as Adam converge for all four of them. For instance, at $\gamma = 0.5$, the plain method diverges, while momentum diverges at $\gamma = 1$. This indicates that the optimal learning rate depends on the gradient descent method.

The sensitivity of initial learning rates are shown clearly in Fig. 10, which shows the number of iterations needed to reach a cost $< 10^{-6}$ compared to the closed-form solution. For convergence to the OLS closed form, the Adam and momentum methods are more flexible with the initial learning rate, while the plain method only converges for a smaller set of initial learning rates. The RMSProp never converges. This suggests that the learning rate became constant close to the minimum and was unable to shrink further. Therefore, the RMSProp will not reach the cost level that we set. At learning rates larger than around 0.05, the Adam and AdaGrad methods are insensitive to the initial learning rate, while momentum becomes insensitive to initial learning rate > 0.2 , with few iterations needed for convergence. All five methods converge with a range of initial learning methods to the Ridge closed-form. Still, the Adam and Momentum methods converge with fewest iterations for selected learning rates, similar to the OLS case. Unlike the OLS case, the RMSProp converges and is insensitive to the initial learning rate in the Ridge case.

A higher initial learning rate generally results in fewer iterations needed for convergence. This effect is greatest at small learning rates, while the number of iterations become insensitive to the learning rate when the learning rate is high enough (e.g. momentum at $\gamma \in (0.2, 0.7)$ for OLS). This insensitivity does not appear for the plain method, however, likely due to the constant learning rate, in contrast to the adapted learning rates in the other methods. The stippled line shows the maximum learning rate based on the Hessian matrix. While the plain method only converges for learning rates lower than this value, the other methods can still converge at higher values and are therefore not affected by this limit. This is likely due to the iteratively changing learning rate.

The range of possible initial learning rates and the optimal learning rate is provided in Table II. We will use the optimal gammas defined here for each method hereafter, unless stated otherwise.

We can also apply the gradient descent method using Lasso regression. Lasso regression cannot be solved using analytical expression, as the regression method includes a penalty that is not differentiable at $\theta_j = 0$. This requires an implementation for the derivative of $\theta = 0$. The automatic differentiation function returns a gradient= 1 at $\theta = 0$, while it returns a gradient= -1 at $\theta = -0.3$ and gradient= 1 at $\theta = 0.3$. We include a soft threshold (Eq. 12) to the gradient descent code, causing the Lasso regression to truncate at zero at coefficients smaller than $\lambda/2$.

Table II: Ranges of initial learning rates γ where the gradient descent converges to a cost $\leq 1e-6$ compared to the closed-form cost (or based on scikit-learn for Lasso). The optimal initial learning rate (γ^*) is also included.

Method	OLS	Ridge	Lasso
Plain	0.0251-0.2512 (γ^* : 0.251)	0.0063-0.2512 (γ^* : 0.251)	0.0631-0.2512 (γ^* : 0.251)
Momentum	0.0025-0.6310 (γ^* : 0.251)	0.0010-0.6310 (γ^* : 0.100)	0.0063-0.6310 (γ^* : 0.251)
AdaGrad	0.0158-1.000 (γ^* : 0.063)	0.0100-1.000 (γ^* : 0.100)	0.0251-1.000 (γ^* : 0.063)
RMSProp	-	0.0025-1.0000 (γ^* : 0.010)	-
Adam	0.0010-1.000 (γ^* : 0.025)	0.0010-1.000 (γ^* : 0.063)	0.0010-1.000 (γ^* : 0.025)

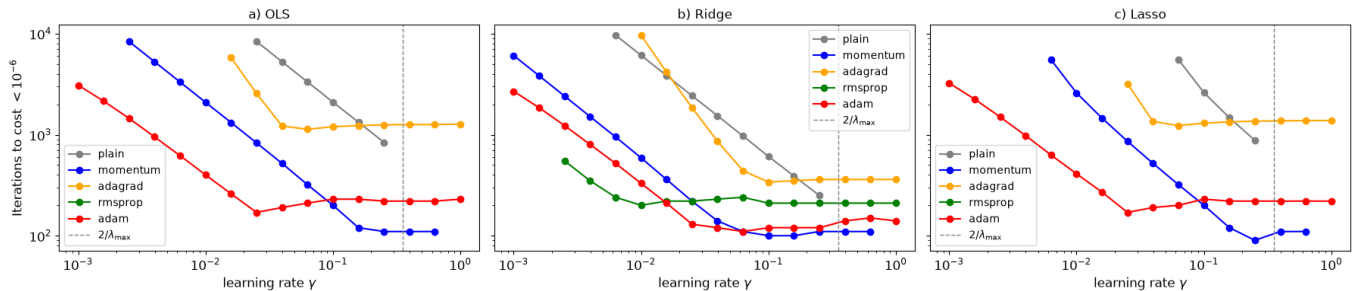


Figure 10: Sensitivity of iterations to convergence for different initial learning rates on the 5 gradient descent methods for (a) OLS, (b) Ridge and (c) Lasso regression. RMSprop (green) never converges to $< 10^{-6}$ in OLS and Lasso, but is almost insensitive to initial learning rate in Ridge. Adam (red) and momentum (blue) are the most flexible in possible initial learning rates and need overall fewer iterations for convergence compared to plain (gray) and Adagrad (yellow). See also Table II for the range of possible learning rates and the optimal learning rate.

A quick analysis showed that a penalty parameter $\lambda = 10^{-5}$ results in the largest range of possible learning rates for the different methods (not shown). This penalty is therefore used hereafter for Lasso regression. Fig. 10 (c) shows the sensitivity of iterations needed for convergence for different initial learning rates using the Lasso regression. The sensitivity is similar to that in the OLS regression, however, the ranges of possible learning rates are slightly smaller. Additionally, the RMSProp never converges to a cost $< 10^{-6}$. This similarity to OLS can be due to the small penalty parameter chosen, as only very small coefficients will be truncated. The ranges and optimal initial learning rates are found in Table II.

Fig. 11 shows the gradient descent methods with the OLS (a), Ridge (b) and Lasso (c) regression methods. As there is no closed form for Lasso, we compare the gradient descent method with the `Lasso` class from scikit-learn [10]. The initial learning rates are the optimal learning rates, found from the sensitivity scan in Fig. 10. As RMSProp does not converge in OLS or Lasso for any learning rate, this method is not shown here. The Lasso regression model differs to OLS and Ridge. The smallest difference to the scikit-learn `Lasso` class is just above 10^{-4} , meaning none of the values converge to the selected tolerance of 10^{-8} . The momentum method reaches its minimum slightly before Adam, however, Adam has a slightly smaller minimum. These two methods are also the methods that reach convergence in the fewest number of iterations in both OLS and Ridge regression. The difference of $> 10^{-4}$ to the scikit-learn `Lasso` class can be explained by the function using coordinate descent in their calculation [10], while we apply the Lasso using a subgradient in the gradient descent method.

At their optimal learning rates, the plain method converges in fewer iterations than AdaGrad, suggesting that the memory becomes very large for the latter method. As the memory in AdaGrad increases with iterations, the learning rate becomes substantially small. This suggests that AdaGrad is not an ideal method when studying a smooth Runge function.

Another gradient descent method is the stochastic gradient descent (Section IID 3). Fig. 12 shows the five methods using stochastic gradient descent for four selected numbers of mini-batches using both OLS and Ridge regression. $M = 100$ is the full-batch gradient descent (equal to the number of data points n). The evaluations are single-point gradient evaluations, which are given by the number of epochs times the number of data points n . Increasing the number of mini-batches reduces the spread of the calculated gradient. However, a higher number of mini-batches is more computationally expensive. The initial learning rates are based on the optimal values from Table II. The plain and momentum methods both diverge for $M = 1$ and $M = 5$, while the other three methods equilibrate. Adam finds its minimum at the fewest evaluations, but keeps oscillating around the minimum. The Ridge regression figure (bottom row) clearly shows troughs for several of the methods, before the distance to the closed form increases and stagnates

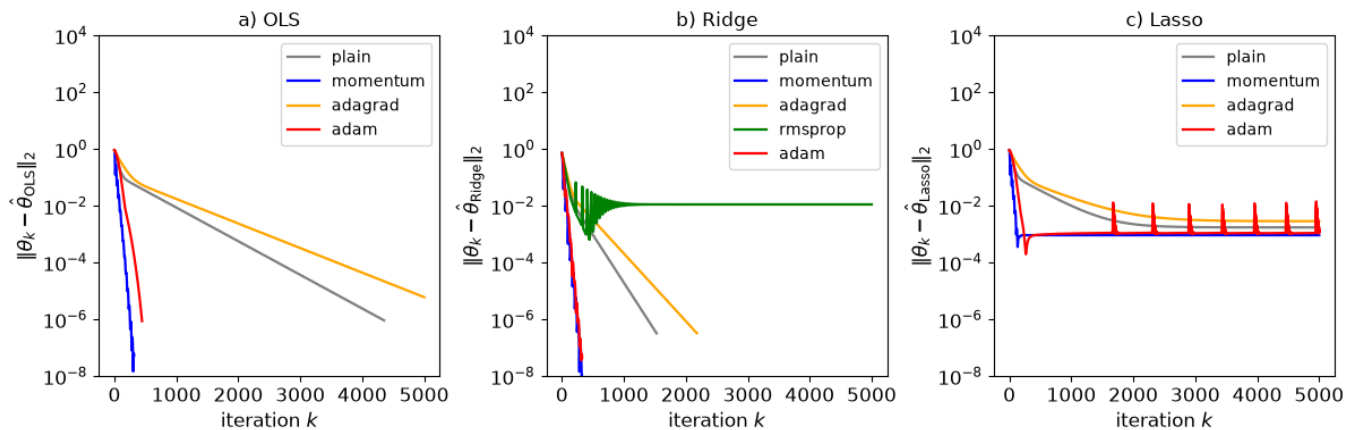


Figure 11: Gradient descent convergence for (a) OLS, (b) Ridge and (c) Lasso. As Lasso does not have a closed-form solution, the convergence is instead compared to scikit-learn’s function. Each method uses their respective optimal initial gamma, which is found in Table II.

at a plateau. This may be due to the absence of a stopping criterion in the function, which would have stopped the gradient descent at the minimum. Instead, the code runs through all epochs no matter where the minimum is, creating unnecessary computational costs.

The size of the plateau is related to the learning rate, where a lower learning rate results in a larger distance to the closed form solution for the plateau. However, if the learning rate is too high, the gradient descent may diverge. The methods reach their troughs and plateaus at a higher number of evaluations for higher mini-batch numbers. This is partly because the noise level from the stochastic batches reduces when we use fewer batches. The full-batch gradient does not reach this plateau that is seen in the other mini-batches, but keeps slowly minimising the distance to the closed-form solution. However, while a full-batch gradient descent produces the lowest distance to the closed form, this will be very expensive for large datasets.

All of the methods equilibrate at a distance 0.01-1 to the closed form (except in the full-batch gradient which is essentially the plain gradient descent). This suggests that a decaying learning rate should be applied to the stochastic gradient descent. Fig. 13 shows the plain and Adam gradient descent methods with and without a learning rate schedule for mini-batch $M = 5$ and $M = 25$. We compare it to the Ridge closed-form solution. We use decaying schedules, selected as the combination of t_0 and t_1 that produce the smallest difference to the closed form (see Fig. 15). Applying a schedule produces closer solutions to the closed-form solution (mainly for $M=5$), and the minimum is found at higher number of evaluations before the gradient descent reaches a plateau. The largest improvement is at $M = 5$.

C. Final model selection

Combining the results from the previous section, we aim to select a model that performs well with the Runge function. We apply the k-fold cross-validation using $k=5$ on the gradient descent methods presented above, using OLS, Ridge and Lasso regressions.

Using stochastic gradient descent results in smaller costs, however, the distance to the closed-form solutions are larger. This is an important trade-off to consider when using large datasets. For this dataset with 100 data points on the 1-dimensional Runge function, we prefer the accuracy from the full gradient descent methods rather than using mini-batches, since their benefit is less significant at this smaller scale.

The cross-validation calculates the MSE for polynomial degrees 1-12 and, in case of Ridge and Lasso, penalty parameters $1e-5$ to 1. We calculate the CV-MSE for all five gradient descent methods using OLS regression, and for Adam using Ridge and Lasso regression. We first choose an initial learning rate across all methods based on Fig. ???. All of the methods converge at $\gamma = 0.1$ (except for RMSProp, which never converges in OLS and Lasso). However, the choice affects the results and this learning rate is chosen mainly to have the same initial value across all methods for comparison. We compare this with the MSE from using the respective optimal initial learning rates (γ^*) as stated in Table II. Note that these learning rates are based on analysis with a 5-degree polynomial and selected penalty parameters (0, 0.01, $1e-5$ for OLS, Ridge and Lasso, respectively) and will therefore also affect the MSE results. The latter also applies to the CV-MSE analysis using Adam gradient descent, which includes the optimal learning rate in

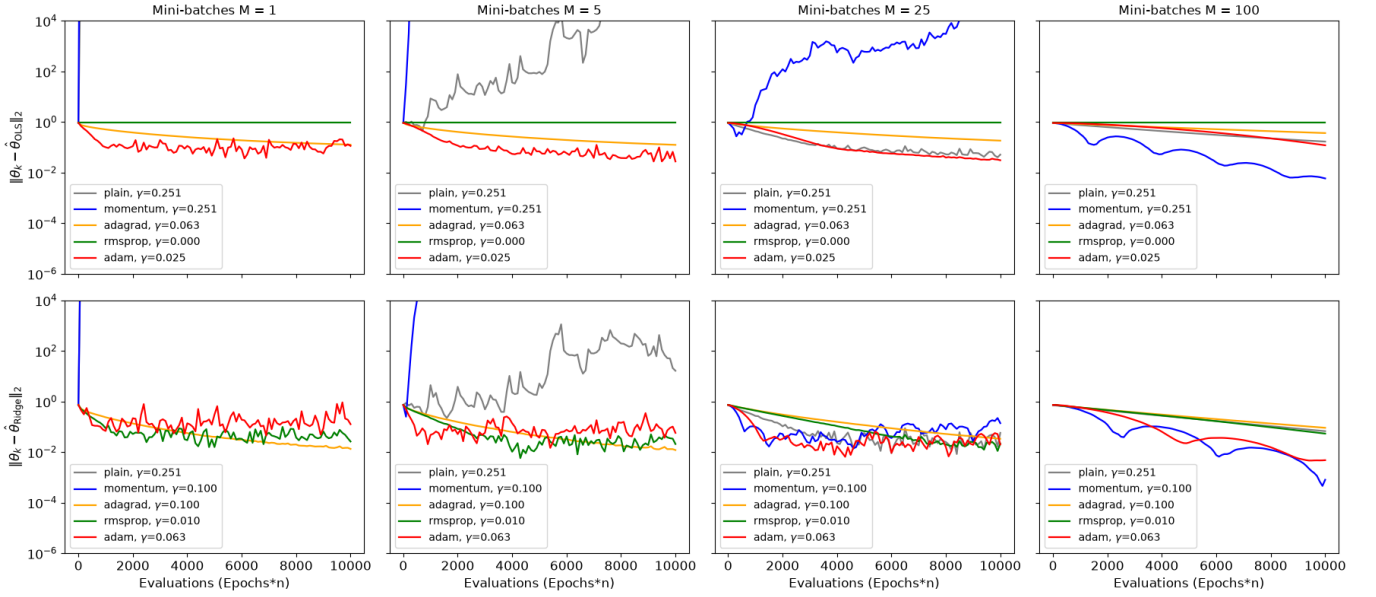


Figure 12: Convergence to closed-form solution with (upper panel) OLS and (lower panel) Ridge, using stochastic gradient descent with plain (gray), momentum (blue), adagrad (yellow), rmsprop (green) and adam (red). Four numbers of mini-batch sizes are included, where $M=100$ is the full-batch gradient descent. The optimal initial learning rates are used for each method.

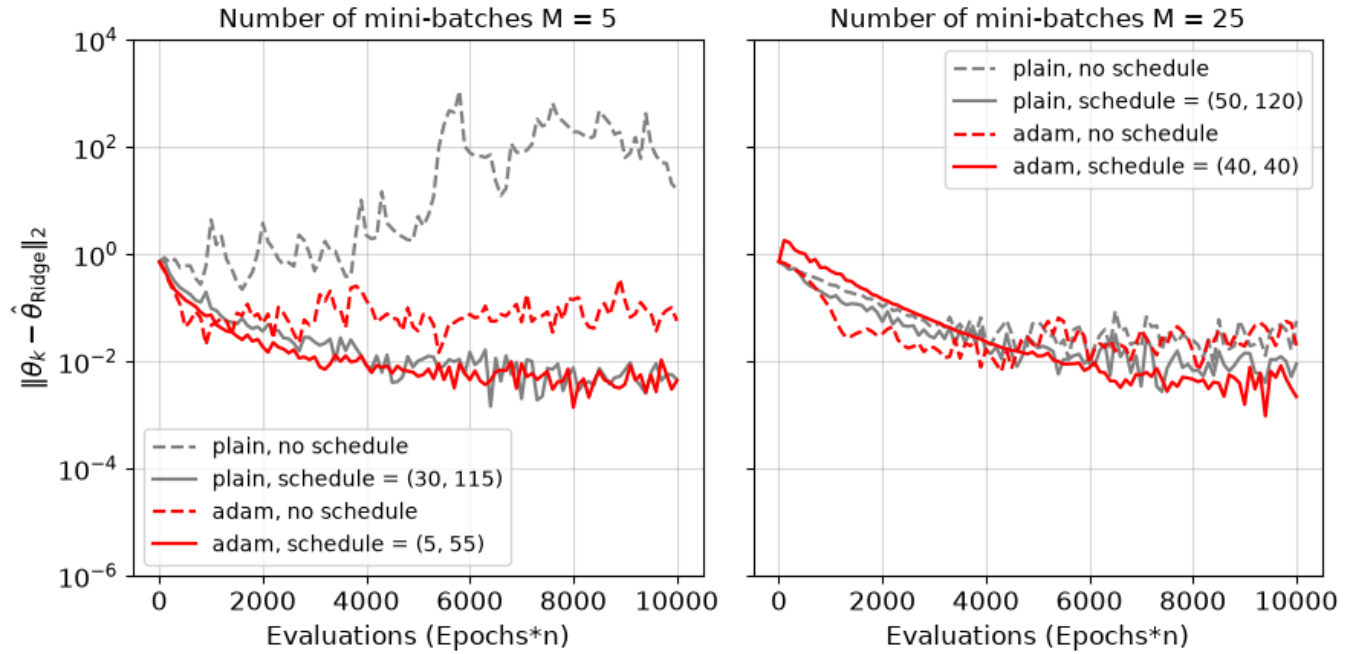


Figure 13: Convergence to closed-form solution with Ridge, using stochastic gradient descent both with (hatched lines) and without (solid lines) a decaying schedule, for plain (gray) and adam (red). The schedule (t_0, t_1) are selected as the combination that produced the smallest distance to the closed-form solution (see Fig. 15). The optimal initial learning rates are used for each method.

the analysis.

Fig. 14 shows that the MSE reduces with increasing polynomial degree and decreasing lambda across the methods. However, there are small differences in MSE at degrees above 4 and penalty parameter $< 1e - 3$ in both Ridge and Lasso. This suggests that high polynomials result in overfitting, and create unnecessary large computational costs. The Runge's function is a good example for this tradeoff. The same analysis with the other gradient descent methods

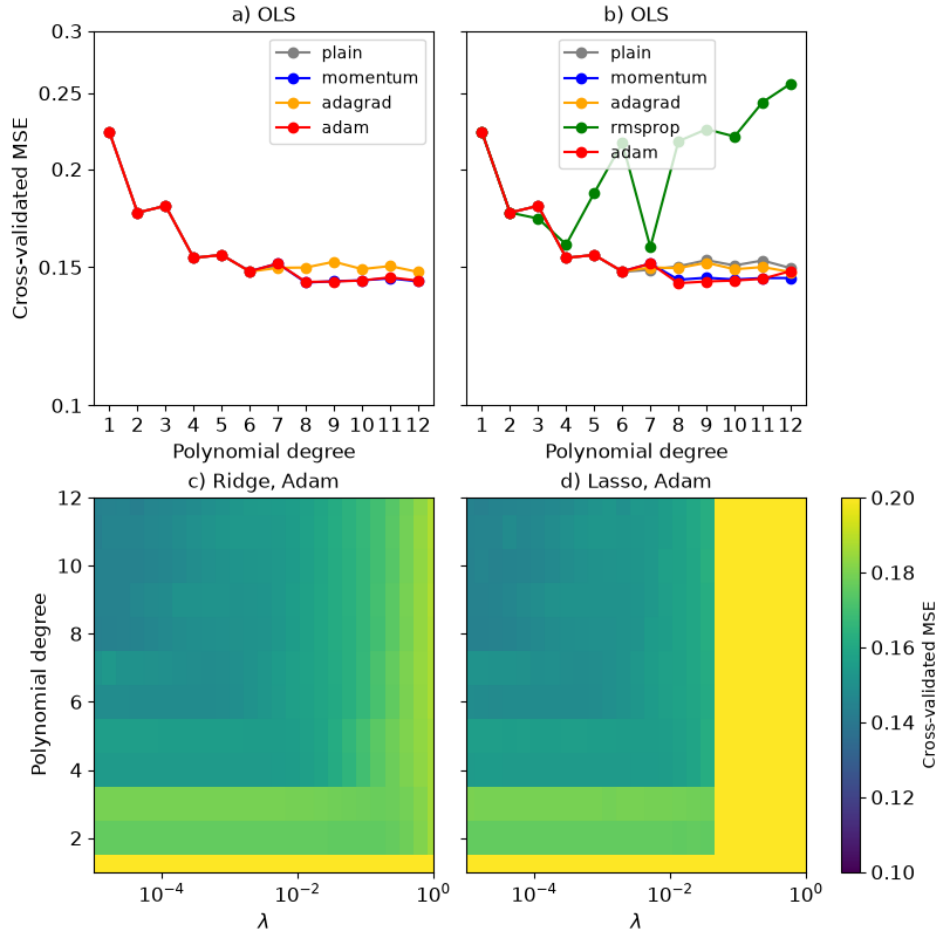


Figure 14: Cross-validated MSE for the gradient descent methods using OLS, Ridge and Lasso regression as a function of polynomial degree and penalty parameter λ . All five gradient descent methods are used with OLS, first (a) with their respective optimal learning rate, then (b) with a selected learning rate for all methods. Adam gradient descent is analysed with (c) Ridge and (d) Lasso regression.

show similar patterns in MSE as functions of degree and penalty (not shown).

We can compare these results with the MSE from the gradient descent methods that include selected parameters from previous analysis. Table III shows the cross-validated MSE, its standard deviation σ and number of iterations to convergence for each of the gradient descent methods. Here, the optimal learning rates have been selected for each model. The differences in MSE is almost negligible across the regression methods, with differences at $1e-4$. The standard deviation is slightly lower in Ridge regression, than in OLS and Lasso. RMSProp in Ridge regression has a slightly smaller MSE, but also a higher standard deviation than the other methods in Ridge. However, such small differences are negligible when selecting the ideal model. Instead, the number of iterations to convergence shows some interesting results. Overall, the gradient descent converges faster with Ridge than with OLS and Lasso, resulting in smaller computational costs. RMSProp does not converge with OLS and Lasso, as it fails to reach a sufficiently small tolerance level. The plain and AdaGrad methods need many iterations to converge compared to momentum and Adam. The plain gradient descent is not ideal, as we want an adaptive learning rate instead of the constant one in plain. Further, AdaGrad is not ideal for this type of smooth function as it stores all of the history of previous gradients, resulting in very small learning rate steps and large memory. Both momentum and Adam converge with fewest iterations compared to the other methods for all three regression types. However, Adam includes the momentum term as well as the RMS term, and is also the least sensitive to the initial learning rate. Therefore, we find that Adam using Ridge regression is the best model for this project with the Runge's function, as it is both quick and flexible. Kingma Ba [6] concluded similarly to us that Adam outperforms Adagrad and RMSProp in their test situations and intercomparisons with large benchmarks describe Adam as one of the most reliable methods across tests [12]. Nevertheless, there are simple convex problems for which Adam does not converge towards the optimal solution

Table III: k-fold cross-validated MSE using gradient descent, based on function with polynomial degree 5.

Regression method	Method	λ	γ	MSE	σ	iterations to convergence
OLS	Plain	0	0.251	0.1555	0.0262	840
	Momentum	0	0.251	0.1555	0.262	110
	AdaGrad	0	0.063	0.155	0.0262	1130
	RMSProp	0	-	-	-	-
	Adam	0	0.025	0.1555	0.0262	170
Ridge	Plain	1e-2	0.251	0.1557	0.0234	250
	Momentum	1e-2	0.100	0.1557	0.0234	100
	AdaGrad	1e-2	0.100	0.1557	0.0234	340
	RMSProp	1e-2	0.010	0.1534	0.0246	200
	Adam	1e-2	0.063	0.1557	0.0234	110
Lasso	Plain	1e-5	0.251	0.1555	0.0262	880
	Momentum	1e-5	0.251	0.1555	0.0262	90
	AdaGrad	1e-5	0.063	0.1555	0.0262	1230
	RMSProp	1e-5	-	-	-	-
	Adam	1e-5	0.025	0.1555	0.0262	170

[13].

IV. CONCLUSION

In this paper, we have investigated polynomial complexity, regularisation and optimisation methods using the Runge function.

The closed forms of OLS and Ridge regression provide the best results of approximating the Runge function at moderate polynomial degrees (4 to 9) and become unstable at higher degrees due to overfitting. Larger sample sizes allow more stable results at high polynomial degrees. Both bootstrap and cross-validation methods reproduce the typical U-shape of the error for varying polynomial degree. This shape is dominated by bias for low degrees and variance at high degrees. In our standard experiment with a sample size of 100, both bootstrap and cross-validation produce good results for polynomial degrees 4 to 9 and lie within the same uncertainty.

We find that the Adam model using Ridge regression with a fifth polynomial degree, an initial learning rate 0.063, and penalty parameter 1e-2 provides the best model. Adam is a versatile method and is flexible with respect to the learning rate. This model converges to the closed-form solutions in few iterations, and is flexible with regards to chosen learning rate and penalties. The closed form is the analytical result and therefore accurate result, but is highly computational, so unrealistic to use with large datasets. This setting is a balance between keeping low computational costs but maintaining predictive accuracy with the model.

The learning rates were scanned using only 16 different values but with a range of 1e-3 to 1, making the intervals between learning rates coarse. A finer interval between the learning rates could result in finding a convergence of the RMSprop. Throughout the analysis, we needed to select specific values for various variables (penalty parameter, learning rate, polynomial degree) to use in further analysis. These values are chosen based on MSE and visual selection. Selecting other values will produce different results and possibly a different recommended model. This is particularly important in the final analysis, where we calculate the cross-validated MSE as a function of polynomial degree and penalty parameter, using results based on previous selections. The comparison between optimisation methods is also dependent on the hyperparameters selected for each method. In addition to the learning rate, parameters such as the momentum and decay coefficients can affect convergence, so the observed differences between methods may change under alternative parameter choices. Another limitation is that we tested these methods on solely one function. Although we expect the results to be representative for most functions, we cannot be sure that the suggestions for model and settings are applicable to other functions. Equally, the results may depend on the particular training data and noise used. Although resampling methods provide an assessment of variability, repeating the analysis with different generated datasets could lead to different estimates of the optimal model and hyperparameters.

Currently, the stochastic gradient descent finds its minimum at few iterations, but keeps iterating through all epochs. Further work can therefore involve including a stopping function in the stochastic gradient descent, to reduce the computational cost by looping through all epochs when the method has converged. Moreover, applying this analysis to other functions can provide insight into how general the observed convergence behaviour is and how sensitive the

different methods are to the choice of problem. We can also start measuring computational time specifically, not just number of iterations, since these two values are not always identical due to different levels of computational cost.

-
- [1] C. Runge, *Zeitschrift für Mathematik und Physik* **46**, 224 (1901).
 - [2] M. Hjorth-Jensen, *Applied data analysis and machine learning: Lecture notes*, <https://github.com/CompPhysics/MachineLearning> (2026).
 - [3] N. Qian, *Neural Networks* **12**, 145 (1999), ISSN 0893-6080.
 - [4] J. Duchi, E. Hazan, and Y. Singer, *Journal of Machine Learning Research* **12**, 2121 (2011).
 - [5] G. Hinton, *rmsprop: Divide the gradient by a running average of its recent magnitude* (2012), URL https://www.cs.toronto.edu/~tijmen/csc321/slides/lecture_slides_lec6.pdf.
 - [6] D. P. Kingma and J. Ba, in *International Conference on Learning Representations (ICLR)* (2015), arXiv:1412.6980.
 - [7] H. Robbins and S. Monro, *The Annals of Mathematical Statistics* **22**, 400 (1951).
 - [8] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, et al., *JAX: composable transformations of Python+NumPy programs*, <http://github.com/google/jax> (2018).
 - [9] C. R. Harris, K. J. Millman, S. J. van der Walt, et al., *Nature* **585**, 357 (2020).
 - [10] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, et al., *Journal of Machine Learning Research* **12**, 2825 (2011).
 - [11] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction, Second Edition. Springer Series in Statistics* (Springer, New York, 2009), URL <https://link.springer.com/book/10.1007%2F978-0-387-84858-7>.
 - [12] R. M. Schmidt, F. Schneider, and P. Hennig, *Descending through a crowded valley - benchmarking deep learning optimizers* (2021), 2007.01547, URL <https://arxiv.org/abs/2007.01547>.
 - [13] S. J. Reddi, S. Kale, and S. Kumar, *On the convergence of adam and beyond* (2019), 1904.09237, URL <https://arxiv.org/abs/1904.09237>.

Appendix A: Appendix A: Extra figures

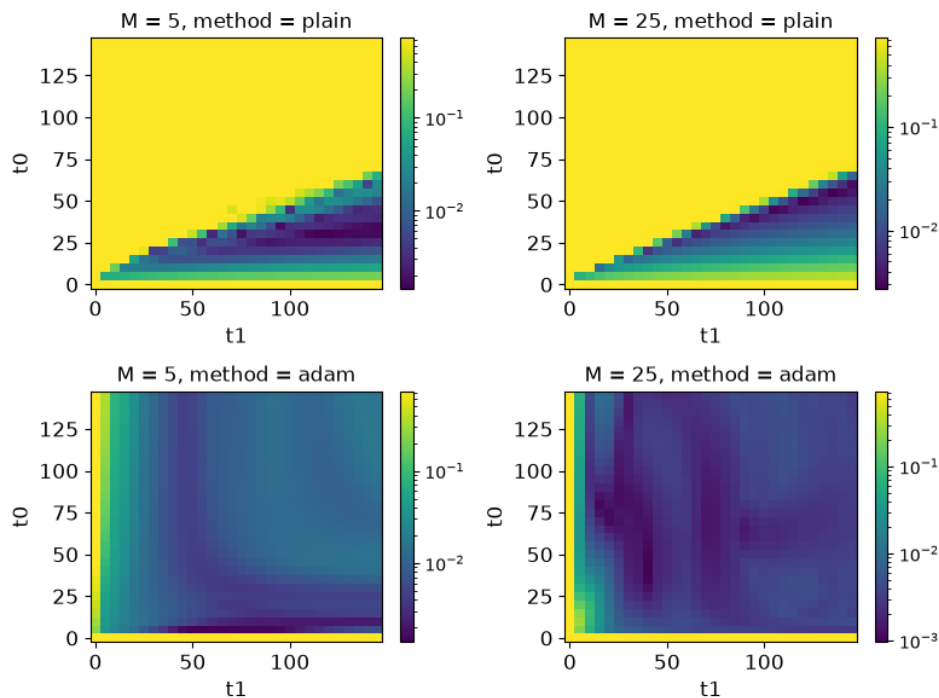


Figure 15: Heatmap of the distance to the closed-form solution for different combinations of t_0 and t_1 to the schedule in stochastic gradient descent.

Appendix B: Appendix B: Derivation of the closed-form OLS solution

Let $\mathbf{y} \in \mathbb{R}^n$ denote the observed values, let $\mathbf{X} \in \mathbb{R}^{n \times p}$ be the design matrix, and let $\boldsymbol{\theta} \in \mathbb{R}^p$ contain the regression parameters. The predicted values are then

$$\hat{\mathbf{y}} = \mathbf{X}\boldsymbol{\theta}. \quad (\text{B1})$$

The MSE can be written in matrix form as

$$C(\boldsymbol{\theta}) = \frac{1}{n}(\mathbf{y} - \mathbf{X}\boldsymbol{\theta})^T(\mathbf{y} - \mathbf{X}\boldsymbol{\theta}). \quad (\text{B2})$$

The optimal parameter vector $\boldsymbol{\theta}_{\text{OLS}}$ is the value that minimises this cost function. We therefore differentiate $C(\boldsymbol{\theta})$ with respect to $\boldsymbol{\theta}$ and set the resulting gradient equal to zero:

$$\nabla_{\boldsymbol{\theta}} C(\boldsymbol{\theta}) = \frac{1}{n} \nabla_{\boldsymbol{\theta}} [(\mathbf{y} - \mathbf{X}\boldsymbol{\theta})^T(\mathbf{y} - \mathbf{X}\boldsymbol{\theta})]. \quad (\text{B3})$$

Expanding the expression inside the brackets gives

$$C(\boldsymbol{\theta}) = \frac{1}{n} (\mathbf{y}^T \mathbf{y} - \mathbf{y}^T \mathbf{X}\boldsymbol{\theta} - \boldsymbol{\theta}^T \mathbf{X}^T \mathbf{y} + \boldsymbol{\theta}^T \mathbf{X}^T \mathbf{X} \boldsymbol{\theta}). \quad (\text{B4})$$

Since $\mathbf{y}^T \mathbf{X}\boldsymbol{\theta}$ is a scalar and therefore equal to its transpose,

$$\mathbf{y}^T \mathbf{X}\boldsymbol{\theta} = \boldsymbol{\theta}^T \mathbf{X}^T \mathbf{y}, \quad (\text{B5})$$

so the gradient becomes

$$\nabla_{\boldsymbol{\theta}} C(\boldsymbol{\theta}) = \frac{2}{n} (\mathbf{X}^T \mathbf{X}\boldsymbol{\theta} - \mathbf{X}^T \mathbf{y}). \quad (\text{B6})$$

At the minimum, the gradient is zero. Hence,

$$\frac{2}{n} (\mathbf{X}^T \mathbf{X}\boldsymbol{\theta}_{\text{OLS}} - \mathbf{X}^T \mathbf{y}) = 0, \quad (\text{B7})$$

$$\mathbf{X}^T \mathbf{X}\boldsymbol{\theta}_{\text{OLS}} = \mathbf{X}^T \mathbf{y}. \quad (\text{B8})$$

Assuming that $\mathbf{X}^T \mathbf{X}$ is invertible, multiplying from the left by $(\mathbf{X}^T \mathbf{X})^{-1}$ gives the closed-form OLS solution

$$\boxed{\boldsymbol{\theta}_{\text{OLS}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}}. \quad (\text{B9})$$

This is the analytical solution used as reference when evaluating the convergence of the gradient-based optimisation methods.

Appendix C: Appendix C: Bias-Variance Decomposition Derivation

Assuming the data comes from the regression model

$$y = f(x) + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2), \quad (\text{C1})$$

Since the training set is random, the fitted prediction $\tilde{y}$ is also a random variable. The expected squared error at a test point x is therefore

$$\mathbb{E} [(y - \tilde{y})^2]. \quad (\text{C2})$$

To decompose this error, add and subtract the expected prediction $\mathbb{E}[\tilde{y}]$:

$$\mathbb{E}[(y - \tilde{y})^2] = \mathbb{E}[(y - \tilde{y} + \mathbb{E}[\tilde{y}] - \mathbb{E}[\tilde{y}])^2]. \quad (\text{C3})$$

Using $y = f(x) + \epsilon$, this becomes

$$\mathbb{E}[(y - \tilde{y})^2] = \mathbb{E}[(f(x) + \epsilon - \mathbb{E}[\tilde{y}] + \mathbb{E}[\tilde{y}] - \tilde{y})^2]. \quad (\text{C4})$$

Expanding the square gives

$$\begin{aligned} \mathbb{E}[(y - \tilde{y})^2] &= \mathbb{E}[(f(x) - \mathbb{E}[\tilde{y}])^2] + \mathbb{E}[\epsilon^2] + \mathbb{E}[(\tilde{y} - \mathbb{E}[\tilde{y}])^2] \\ &\quad + 2\mathbb{E}[(f(x) - \mathbb{E}[\tilde{y}])\epsilon] \\ &\quad - 2\mathbb{E}[(f(x) - \mathbb{E}[\tilde{y}]) (\tilde{y} - \mathbb{E}[\tilde{y}])] \\ &\quad - 2\mathbb{E}[\epsilon (\tilde{y} - \mathbb{E}[\tilde{y}])]. \end{aligned} \quad (\text{C5})$$

Since $\mathbb{E}[\epsilon] = 0$,

$$\mathbb{E}[(f(x) - \mathbb{E}[\tilde{y}])\epsilon] = (f(x) - \mathbb{E}[\tilde{y}])\mathbb{E}[\epsilon] = 0. \quad (\text{C6})$$

Similarly, by the definition of the expected prediction,

$$\mathbb{E}[\tilde{y} - \mathbb{E}[\tilde{y}]] = 0, \quad (\text{C7})$$

so

$$\mathbb{E}[(f(x) - \mathbb{E}[\tilde{y}]) (\tilde{y} - \mathbb{E}[\tilde{y}])] = 0. \quad (\text{C8})$$

so

$$\mathbb{E}[\epsilon (\tilde{y} - \mathbb{E}[\tilde{y}])] = \mathbb{E}[\epsilon]\mathbb{E}[\tilde{y} - \mathbb{E}[\tilde{y}]] = 0. \quad (\text{C9})$$

Finally, since $\mathbb{E}[\epsilon^2] = \sigma^2$, the expected squared error can be written as

$$\mathbb{E}[(y - \tilde{y})^2] = \underbrace{(f(x) - \mathbb{E}[\tilde{y}])^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}[(\tilde{y} - \mathbb{E}[\tilde{y}])^2]}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{Noise}}. \quad (\text{C10})$$

This decomposition illustrates the bias-variance tradeoff: increasing model complexity may reduce bias while increasing variance, whereas simpler models may have higher bias but lower variance.

Appendix D: Appendix D: Use of AI/LLM tools

A.1 Tools used: Claude (claude.ai, September 2026); GitHub Copilot (VS Code, throughout); Writeful (included in overleaf).

A.2 Text writing and editing.

No AI tools were used for text writing and editing. *Writeful* AI paraphrasing suggestions are a default in overleaf, but we tried to ignore them. Claude AI has been used to debug latex code in one instance, where `\usepackage[table]{xcolor}` creates errors in tables when no colouring is selected (25.09.2026 13:41 CET).

A.3 Code generation and assistance.

Table IV: AI use in coding (0=None, 1=Debugging, 2=Snippet, 3=Skeleton, 4=Substantial).

File & Code block	Level	Description
all	1	GitHub Copilot inline suggestions are a default setting and where occasionally accepted.
wip_project1.ipynb, task a-d)	1	discovered that a variable hadn't been updated
wip_project1.ipynb, task i)	1	Looked into why the different methods did not produce similar results.
readme.md	4	fully AI generated readme file
wip_project1.ipynb	1	removing unnecessary variables and double defined functions to increase reproducibility